

Part 1: Multiply-Accumulate Computation Unit

AI Policy Reminder

Reminder: please see the project AI Policy in Appendix A of the Project Overview document, which explains how AI may and may not be used in this project, and how you should log and disclose AI usage.

Updates

This document has been revised since its initial release. Changes are listed here and shown in **red** in the text.

- **9/21:** Clarified the behavior of `Q` when `input_valid` and `init_acc` are both asserted. A few new sentences added in Section 1.5, at the end of the waveform discussion in Section 2, and in the discussion of Figure 5 in Section 3.

Multiply and accumulate (MAC) is a basic operation used commonly in many different types of computations. MAC is defined as

$$f = f + a \cdot b \quad (1)$$

where a and b are inputs to the system and f is the accumulated value. Notice how the MAC operation is used in convolution. In the convolution pseudocode (see the Project Overview document), we have:

```
Y[r][c] += X[r*S+i][c*S+j] * W[i][j]
```

In Part 1, you will design a MAC module that computes Equation 1. Later parts of the project will then use this MAC module to perform convolution.

A hardware system that performs MAC will require *feedback* because each newly computed f depends on the previous value of f . This process of adding a sequence of numbers into a single running sum is called “accumulation.” The f value will be stored in a register called the *accumulator*.

In this part of the project, you will design and test two versions of a MAC unit: an unpipelined version and a pipelined version. Both versions compute the same thing and share the same interface; they differ only in how the work is divided across clock cycles, and therefore in their timing.

The document is organized in the following way. Section 1 specifies the MAC unit’s overall behavior. (Section 1 is applicable to both the unpipelined and pipelined designs.) Section 2 and Section 3 then describe the two designs you will build (unpipelined and pipelined, respectively). Section 4 describes verifying your system with the testbench we provide. Finally, Section 5 and Section 6 cover what to submit and what to include in your Final Report.

Store all of your files for Part 1 in a subdirectory called `part1/`, and please use the file names and module names specified below.

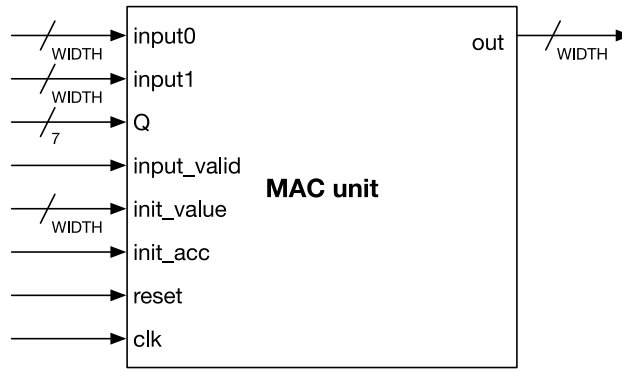


Figure 1: Top-level view of MAC unit's input and output ports

1. MAC Unit Specification

This section describes the MAC unit and its requirements. The contents here apply to both the unpipelined and pipelined versions; more details about those individual designs are given later in Section 2 and Section 3 below.

1.1. Interface

Both versions of the MAC unit have the same top-level ports, which are shown in Figure 1 and described below.

- Input signals `input0` and `input1` correspond to a and b in Equation 1 above. Each will be a signed value that is `WIDTH` bits wide (where `WIDTH` is a parameter in your module).
- 1-bit input signal `input_valid` will be used to tell the module when a new valid set of `input0`, `input1`, and `Q` are provided on the input ports. (`Q` is defined below.) The MAC unit will use the `input_valid` signal to determine when it should update its internally stored values based on the current inputs. (If the inputs are not valid, then they should not cause the accumulator register to change.)
- The output signal `out` is a `WIDTH`-bit signed value representing the MAC output.
- Internally, your MAC unit will accumulate (that is, store the internal value of f) using more than `WIDTH` bits so that it can hold intermediate values without overflowing. Your accumulator width is set by the parameter `ACCW`.¹
- Because `ACCW` will be larger than `WIDTH`, the accumulator will be wider than the MAC's output. This means that the accumulated value must be *scaled down* before it can leave the module. Input `Q` is a 7-bit unsigned value that specifies how far to scale it: the `ACCW`-bit accumulator value is arithmetically right shifted by `Q` bits and then saturated to fit within the `WIDTH` bits of `out`. (Seven bits is enough to represent any useful shift amount, since the largest `ACCW` we allow is 80.) The behavior of the shifting is specified fully in Section 1.3 below.
- We also define a way to *initialize* the MAC unit's accumulator register. (For example, when you want to start a new calculation, you need to initialize it, so you don't continue to accumulate your new results on top of your old one.) Input signal `init_acc` (active high) tells your MAC unit to initialize

¹Please note that here in Part 1, `ACCW` will be a parameter, but in later parts of the project (Parts 3 and 4), your code will calculate `ACCW` automatically given `WIDTH` and `K`. You don't need to worry about that aspect of this now; see Part 3 later for more information.

the accumulator. The `init_value` input (described next) will tell you what value to initialize the accumulator to.

- Input signal `init_value` gives the initial value that the accumulator should take when it is initialized. If `init_acc == 1` on a positive clock edge, then the accumulator register should be set to `init_value`. This is a signed `WIDTH`-bit value, and it must be *sign extended* into the wider `ACCW`-bit accumulator register. (If `init_acc == 0`, then `init_value` is ignored.)
- The `clk` input is a standard clock signal.
- The `reset` input is an active-high, synchronous reset. (That is, your design will reset when `reset==1` on a positive clock edge.) More details about the reset behavior are given in Section 1.2.

Both designs use the same parameters and the same ports. Your unpipelined module (Section 2) must be named `mac`, and your pipelined module (Section 3) must be named `mac_pipe`. Apart from that module name, both must declare their parameters and ports exactly as follows:

```
module mac #(
    parameter WIDTH = 16,
    parameter ACCW = 48
)(
    input signed [WIDTH-1:0] input0, input1, init_value,
    input [6:0] Q,
    output logic signed [WIDTH-1:0] out,
    input clk, reset, init_acc, input_valid
);
```

Or for Part 1.2:

```
module mac_pipe #(
    // ... then continue the same as above
```

It is very important that your modules match this input/output specification exactly, or they will fail the tests we run on your submitted code.

Your designs must work across the full range of legal parameter values:

$$2 \leq \text{WIDTH} < 32 \quad \text{and} \quad 2 \cdot \text{WIDTH} \leq \text{ACCW} \leq 80$$

1.2. Datapath and Accumulator

Internally, the MAC unit is built from a multiplier, an adder, and the accumulator register, followed by logic that scales the accumulated value down to the module's output width. Figure 2 shows this datapath in its simplest form, where every operation happens in a single clock cycle. This is the design you will build in Part 1.1. (The pipelined design of Part 1.2 adds a register but otherwise functions in the same way.) Everything else in this specification applies to both designs.

Clock and reset signals are omitted from this figure but will be needed in both designs. Assume that `input0` and `input1` are each `WIDTH`-bit signed values. The output of the multiplier is a `2*WIDTH` bit signed value. The adder's top input, the adder's output, and the accumulator register are signed `ACCW`-

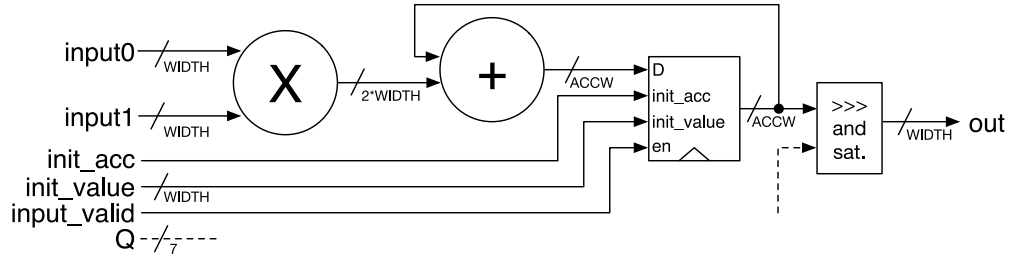


Figure 2: Unpipelined MAC unit. (This figure intentionally omits details about how the *Q* input connects to the shift-and-saturate unit. See Section 1.5 below.)

bit signals. (The multiplier's $2 \times \text{WIDTH}$ -bit product is sign extended to ACCW bits at the addition. As long as your signals are declared signed, SystemVerilog will do this for you when you add the signals with the `+` operator.) The accumulator's output also feeds the shift-and-saturate unit, described in Section 1.3. The shift-and-saturate unit's WIDTH -bit output becomes the MAC module's top-level output `out`, a signed WIDTH -bit value.

Assume the reset is active-high and synchronous: when `reset==1` on a positive clock edge, the register resets to 0. (This will be true of all registers in your project.)

The accumulator register has two synchronous control inputs: `init_acc` and `en`. When `init_acc` is asserted on a positive clock edge, the register initializes to the value given on the `init_value` input. Otherwise, when `en` is asserted on a positive clock edge, the value on the register's D input is stored. (Note the priority: `init_acc` takes precedence over `en`. Reset has the highest priority of all.) How these two control signals are driven differs between the two designs, and is discussed in Section 2 and Section 3.

One other consideration: your register's data input and output are ACCW bits, but your `init_value` signal is WIDTH bits. This means that you must be careful that negative values are *sign extended* in the register. For example, imagine that $\text{WIDTH}=3$ and $\text{ACCW}=6$. Now, if you have `init_value == -3 = 3'b101`, you need the register to store `6'b111101` (which is -3 in 6-bit two's complement). Fortunately, as long as your signals are all correctly defined as signed in your SystemVerilog code, this will happen automatically. If you have problems with initializing the register to negative numbers, check this first. (You could also choose to explicitly write sign extension logic, but this isn't necessary if you define things correctly.)

1.3. Output Shifting and Saturation

The accumulator holds ACCW bits, but `out` is WIDTH bits wide. This pattern is commonly seen in real machine learning accelerators: larger values are internally accumulated but are reduced to narrower representations before leaving the computational block. Why does this happen? Because large data widths are expensive in terms of memory capacity and bandwidth, register size, and even the numbers of wires needed on chip. (Imagine a large parallel system with thousands of parallel compute units; the difference between 8 bit outputs versus 48 bit outputs is huge when repeated thousands of times!) So, accelerators for quantized neural networks solve this the same way we do here: by using enough bits **within** the accumulator to correctly perform the desired computations, and then reducing the number of bits (and thus the precision) of the final computed result.

Your MAC unit will perform this process in two steps: shifting and saturation. These steps are performed as combinational logic in the shift-and-saturate unit, shown as the box labeled `>>> and sat.` in Figure 2.

Step 1: Arithmetic right shift by Q. First, the ACCW-bit accumulator value is shifted right by Q bits. This must be an *arithmetic* shift (`>>>` in SystemVerilog on a signed value), which copies the sign bit into the most significant bit positions.

For example, `4'b1011 >>> 1 = 4'b1101` and `4'b0011 >>> 1 = 4'b0001`. Note that the shift truncates the least significant bits, discarding the bits shifted out. This is equivalent to rounding toward $-\infty$, not toward zero.

For example:

- If accumulator = 99 and Q = 3: then `99 >>> 3 = 12` (not 12.375)
- If accumulator = -99 and Q = 2: then `-99 >>> 2 = -25` (not -24.75 and not -24)

The shifted value represents the accumulator divided by 2^Q and rounded down. So in the first example, the wide accumulator calculated 99; after shifting we have 12; we interpret this as $12 \times 2^3 = 96$. Or the second example: $-25 \times 2^2 = -100$. (In both cases, we get a small amount of rounding error.)

In real machine learning systems, how is Q determined? It depends on what computation is being performed and how many accumulations are done. A larger neural network layer will require more accumulations, potentially leading to larger-magnitude accumulator values, and therefore larger Q. Smaller layers will accumulate fewer times, requiring lower Q values. Quantized neural networks are created with this question in mind; profiling software is used to observe the model's behavior and optimize the values of Q needed for each layer.

Step 2: Saturation. What happens if, during Step 1, the Q value is chosen to be too small? The result would be that even after right-shifting the accumulator's output, the result *still* doesn't fit within WIDTH bits. In this case, *saturation* is the safety net, clipping the shifted value down to the largest positive or negative value that fits in the WIDTH bit output. When saturation happens, information is lost, but it gives the closest WIDTH bit approximation of the desired value.

For example, let WIDTH=8. A signed 8-bit value can be in the range of `8'b10000000` to `8'b01111111`, or -128 to +127. Suppose that the accumulator holds value 200:

- If Q=0: then `200 >>> 0 = 200`, which is greater than 127, so out **saturates** to the largest positive value it can fit, 127.
- If Q=1: then `200 >>> 1 = 100`. This fits in 8 bits **without saturating**, so the result is out = 100.

Or suppose the accumulator holds -200:

- If Q=0: then `-200 >>> 0 = -200`. Since this is less than -128, the result **saturates**, giving out = -128.
- If Q=1: then `-200 >>> 1 = -100`. Since this is greater than -128, it fits in 8 bits **without saturating**, so the result is out = -100.

The following definitions may be useful when you want to implement saturation. Feel free to copy/paste these two lines directly into your module:

```
localparam signed [ACCW-1:0] MAXOUT = (1 <<< (WIDTH-1)) - 1;
localparam signed [ACCW-1:0] MINOUT = -(1 <<< (WIDTH-1));
```

Then your code can use MAXOUT and MINOUT to refer to the maximum and minimum values of the output.

1.4. Accumulator Overflow

The scaling and saturation logic described above takes care of reducing the ACCW bit accumulator value down to the WIDTH bit MAC output. However, one remaining challenge is: what happens if the accumulated value grows larger than we can fit in the ACCW bits of the accumulator register? This is called *overflow*.

Overflow occurs when the result of an arithmetic operation cannot be represented using the allotted number of bits. You can tell that an adder has overflowed if:

- you add two positive values and get a negative answer, or
- you add two negative values and get a positive answer.

For example, let WIDTH=4 and ACCW=8. A signed 8-bit value can be in the range of 8'b10000000 to 8'b01111111, or -128 to +127.

Now imagine that your accumulator is adding two numbers: 106 + 64. Obviously, the sum (170) is too large to fit within the 8-bit signed accumulator. It would require 9 bits (9'b010101010), but since ACCW is only 8, the value will **overflow**, and the top bit will be lost, leaving 8'b10101010 or -86. Two positive numbers were added, and the value grew too large to fit, so it overflowed back to a negative number. Remember, this occurs at the *accumulator*, so this result will then get shifted and saturated as described in Section 1.3.

Overflow is **more destructive** than saturation because this wrap-around behavior (negative numbers becoming positive or positive numbers becoming negative) results in much larger errors. Nevertheless, in Part 1 your hardware does not need to do anything special about it: just make the accumulator register ACCW bits wide and let it wrap around naturally. (Our testbench in Section 4 expects this behavior.) ***Do not confuse this with the output saturation described above.*** These are two separate things, and your design needs both behaviors:

- The ACCW-bit **accumulator** wraps around (overflows) naturally if the accumulated value is larger than ACCW bits. Nothing detects or prevents this in Part 1.
- The WIDTH-bit **output** saturates if the shifted result is larger than WIDTH bits. This behavior is required, and it is applied to whatever value the accumulator produces.

Accumulator overflow can be a challenge in real systems. (Think about it like this: if your convolution should give you a large positive value and your MAC unit outputs a large negative value, that's a huge error.) In practice, real systems handle this by simply ensuring that ACCW is *big enough* to prevent overflow given the computations supported by the system. We will handle this in the same way. In Part 3, where you will use this MAC module as part of a larger convolution system, we know the dimensions of the matrices being convolved. This allows us to *automatically* choose ACCW to be large enough to guarantee the accumulator cannot overflow. (You don't need to worry about this here. It is explained in more detail in Part 3, where you will use it.)

1.5. Timing of the Q and input_valid Signals

Recall from Section 1.1 that input_valid is used to indicate when the input signals input0, input1, and Q are valid. Although this seems straightforward, there is a little bit of complexity related to the timing of these signals that you need to consider in your designs.

First, let's think about Q, which specifies how many bits the accumulator value is right shifted by to produce the output. Importantly, the value of Q is presented at the *same time* as the input0 and input1

it applies to. Because there is a register between the inputs and the shifting logic, you cannot directly connect the Q input to the shifter. If you did, Q would arrive at the shifter's input *before* the corresponding accumulation of $\text{input0} * \text{input1}$. So the **current** value of Q must not affect the output immediately. Rather, it must affect the output ***alongside the input values with which it arrived***. Think carefully about how you must delay the Q signal to arrive at the shifter at the right time.

A similar concern will arise with the `input_valid` signal in your **pipelined** design (Part 1.2), but this is not an issue in the unpipelined design of Part 1.1. We will revisit this when introducing the pipelined design in Section 3 below.

The general rule for both signals: think of Q and `input_valid` as traveling through your module together alongside the data they arrived with. Wherever that data is delayed by a register, they are delayed by a register too.

The `init_acc` signal does not affect Q. Your Q register(s) should be enabled by `input_valid` (and its delayed copies) alone. If `init_acc` and `input_valid` are both asserted on the same clock edge, the accumulator is initialized, and the Q from that cycle is still captured and applied to the newly initialized value.

2. Part 1.1: Unpipelined MAC

Your first task for Part 1 is to build the single-cycle unpipelined MAC unit shown in Figure 2. Implement it, test it thoroughly in simulation, and synthesize it.

In this design, the accumulator is the only register on the datapath, so everything from the input ports through the multiplier and adder is combinational. This gives the design a latency of one clock cycle: a set of inputs presented before a positive clock edge affects out immediately after that edge.

Because of this, the accumulator register's control signals need no adjustment. As shown in Figure 2, the register's `init_acc` signal can be connected directly to the module's `init_acc` input, and the register's `en` signal can be connected directly to the module's `input_valid` input: the product of a valid input pair reaches the accumulator's D input on the very same cycle that `input_valid` is asserted.

The Q input is the one signal that needs additional attention. As explained in Section 1.5, Q arrives alongside the input values it applies to, but the actual shifting by Q happens **on the other side of the accumulator register**. This means that Q cannot be wired directly to the shifter's input without a delay.

Consider the timing waveform in Figure 3. This figure shows example inputs, as well as `product` (the combinational output of the multiplier), `accum_out` (the output of the accumulator register), and `out` (the output of the shift-and-saturate unit).

- Before positive clock edge 1, we see the inputs are 1 and 2 and Q is 0. We interpret this as saying “multiply 1 times 2, add the product to whatever is stored in the accumulator, and then shift that value by $Q=0$ places.”

This means that *one cycle later*, the accumulator becomes $0 + 1*2 = 2$, and `out` (the output of the shift-and-saturate unit) is $2 \ggg 0 = 2$.

- The second cycle has inputs 3 and 4, and $Q=1$. This means one cycle later the accumulator is $2 + 3*4 = 14$ and `out` is $14 \ggg 1 = 7$.

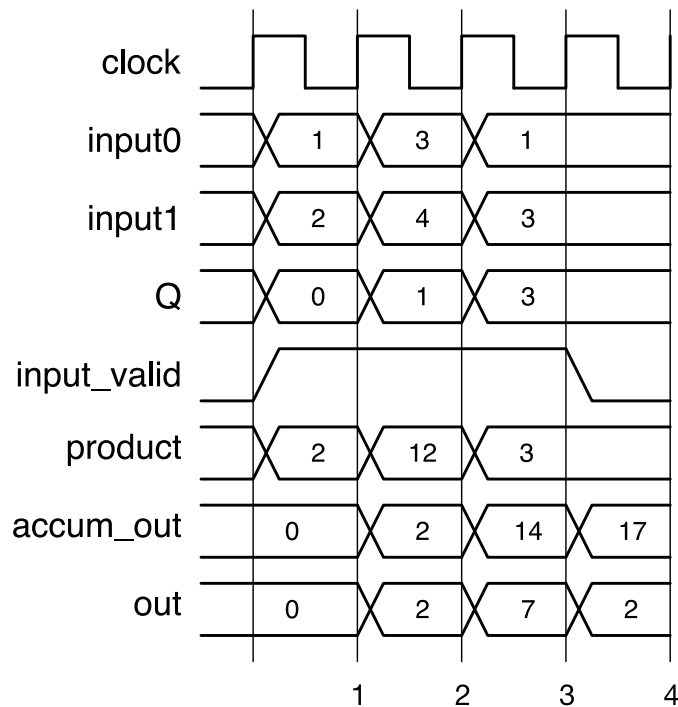


Figure 3: Timing waveform illustrating inputs and Q. Note how the Q value doesn't affect the output immediately; it "travels with" its associated input.

- The third cycle has inputs 1 and 3, and $Q=3$. This means that one cycle later the accumulator is $14 + 1*3 = 17$ and out is $17 \ggg 3 = 2$.

The tricky aspect of this: notice that the shift amount is the value that was Q *at the time the corresponding inputs arrived*, one cycle before the shift happens. At clock edge 1, Q is 0, but the shift by 0 happens in the next cycle. Similarly, Q is 1 at clock edge 2, but the shift by 1 place happens one cycle after that.

Think carefully: how should your system account for this requirement? (Hint: it needs only one small additional component.) Also, don't forget that the Q input only matters when input_valid is 1. (If input_valid is 0, keep the prior valid value of Q.) **This is independent of init_acc. (When input_valid==1, the value of Q should be accepted, even if init_acc is also 1.)**

2.1. Implement and Simulate

Implement this design in a file called `mac.sv`, using the module name, parameters, and port declarations given in Section 1.1.

Use the provided testbench to simulate your design with a variety of parameter values, drawn from the legal ranges given in Section 1.1. (You can't exhaustively check *all possible* combinations of these, but you should try several combinations and cover the extreme values.) See Section 4 below for more information about the testbench we provide.

2.2. Synthesize

Use Synopsys DesignCompiler to synthesize your unpipelined design with `WIDTH=16` and `ACCW=48` and determine the fastest clock frequency your design can reach. Save all your synthesis output files, because you will need them below in Question 1. (I recommend saving each synthesis output file with a name that includes the parameters you set and the clock period you are targeting. This way you can easily go

back and find the results later.) Adapt the synthesis scripts you used in HW2. Don't forget to configure the script with your top-level module name, clock frequency, and so on. If your design uses multiple .sv files, please see the instructions in the comments on lines 9 and 10 of runsynth.tcl. Do not include testbenches when synthesizing.

When you synthesize, make sure you are correctly setting the parameter values in your top-level mac module. The params.sv file used in simulation only affects the testbench; to set the values in synthesis, you must make sure the default parameter values in your top-level module are set correctly.

Find the fastest possible frequency and save the synthesis report from that frequency in a plaintext file. Submit this clearly labeled synthesis report (as a plaintext file) with your code. I recommend naming it synth_mac.txt and using a similar filename convention for future synthesis reports. Include this synthesis report with your Part 1 submission.

Note: your Final Report will require more synthesis data. I recommend you collect that data at this time. Please see the Final Report description below.

It is very important that you correct any synthesis problems reported by DesignCompiler. If you have errors, the tool's output will not be correct. You also must be certain to fix any inferred latches from your design.

One common synthesis warning that you can safely ignore is:

```
Warning: ./file.sv:182: unsigned to signed assignment occurs. (VER-318)
```

If you have questions about other errors or warnings, please see the Topic 5 slides, and follow the instructions from the end of Topic 5 to use the “man” command in DesignCompiler to read the corresponding manual entries.

3. Part 1.2: Pipelined MAC

Your second task for Part 1 is to pipeline this system by inserting a register between the multiplier's output and the adder's input, as shown in Figure 4. If pipelined correctly, you will be able to reach a higher clock frequency because the pipeline register splits up the critical path. See Topic 6 of our class for a more comprehensive discussion of pipelining. In your full convolution system (which you will create in Part 3), you will use this pipelined MAC unit.

Adding this register changes the design's latency from one clock cycle to two: it now takes two clock edges between when input0 and input1 arrive and when they can affect out. This change also affects

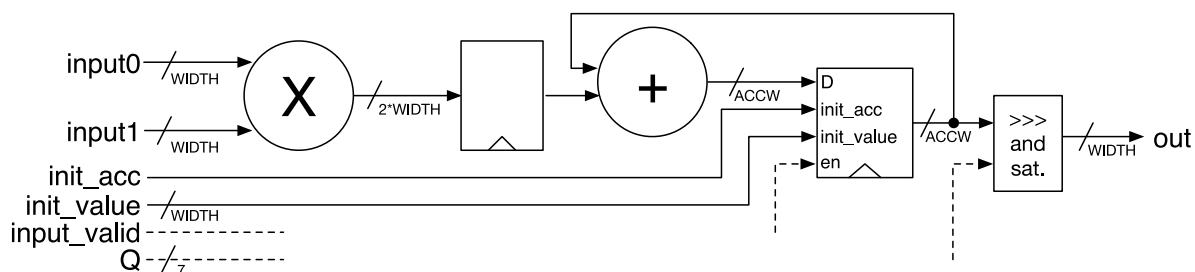


Figure 4: Pipelined MAC unit. (Note that the accumulator's enable signal and the alignment of the Q input are not fully specified.)

how you control the system. Figure 4 doesn't show how the accumulator's enable signal or the shifter's Q signal are controlled. Think carefully about how you solved the Q timing problem in the unpipelined design. Here you have the same problem, but with one added wrinkle: there is now an extra cycle of delay between the inputs and the accumulator. Think carefully about how to keep both the accumulator's enable signal and the Q signal aligned with their associated data. Both will require small changes relative to your unpipelined design.

Notice, however, that `init_acc` connects directly to the accumulator register without any extra delay. This means that initializing the accumulator still happens in a single clock cycle, even though the datapath now has a latency of two. The result is that you can simultaneously ask the MAC unit to initialize the accumulator *and* provide new values of `input0` and `input1`. Consider the diagram shown in Figure 5. (Here, assume that `pipe` represents the output of the pipeline register, `accum_out` represents the output of the accumulator register, and `out` is the output of the shift-and-saturate unit.)

- At first, the system proceeds normally. For example, at clock edge 2, the valid inputs are 1 and 1 with $Q=0$. This means `pipe` becomes 1 after clock edge 2, and after clock edge 3, `accum_out` becomes 1 and `out` becomes $(\text{accum_out} \ggg Q) = (1 \ggg 0) = 1$.
- As another example, at clock edge 3, the valid inputs are 2 and 3 with $Q=1$. So, after clock edge 3, `pipe` becomes 6. Then after clock edge 4, `accum_out` becomes $1+6=7$ and `out` becomes $7 \ggg 1 = 3$.
- Pay careful attention to what happens at clock edge 5 (highlighted green in Figure 5). At this edge, `init_acc` is 1. So, after the clock edge, `accum_out` is initialized to `init_value=19`. But note that this doesn't invalidate the values of `input0`, `input1`, and `Q` provided at the same time (3, 7, and 1). The product $3*7=21$ is still stored on the pipeline register at clock edge 5, and it affects the accumulator and output after clock edge 6 (computing $\text{accum_out} = 19 + 3*7 = 40$ and $\text{out} = 40 \ggg 1 = 20$). So,

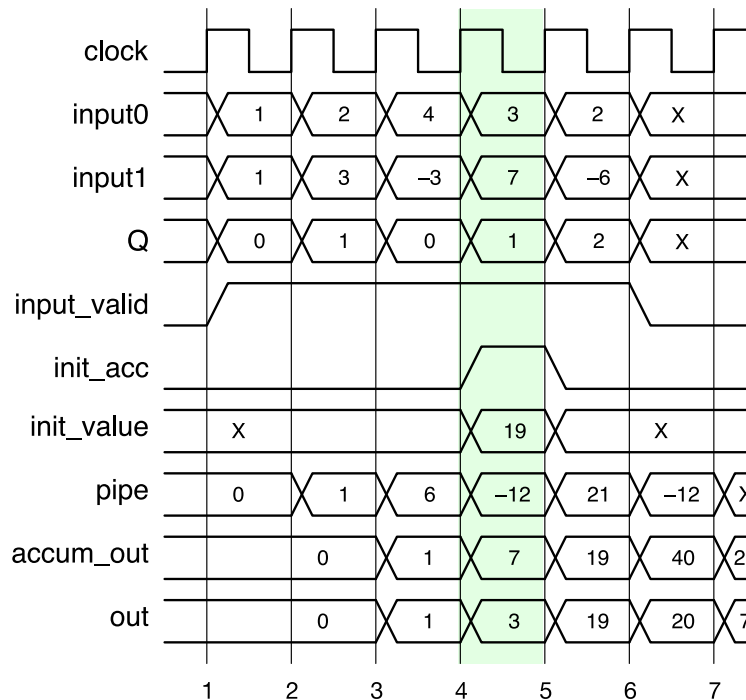


Figure 5: Demonstration of the timing of accumulator initialization.

the inputs at clock edge 5 instruct the MAC unit: “initialize the accumulator to 19, add $3 * 7$ to it, and scale the resulting value by 1,” which results in output 20 after clock edge 6.

- Notice also that immediately after clock edge 5, $out = 19 \ggg 0 = 19$. The shift amount 0 is the Q that arrived at clock edge 4, alongside the product $4 * (-3)$ that `init_acc` discarded. Although that product never reached the accumulator, its Q still advanced through your delay registers and was applied to the initialized value 19. (If your design were to disable the Q register when `init_acc` is asserted, it would incorrectly produce $19 \ggg 1 = 9$ here.)

Note the contrast with the unpipelined design: there, a valid input presented on the same cycle as `init_acc` is discarded, because its product would reach the accumulator on the same cycle that `init_acc` takes effect, and `init_acc` has priority. (Only the product is discarded, though: the Q presented on that cycle is still captured, since `input_valid` was 1, and it scales the newly initialized accumulator value.) This is one rule (`init_acc` takes effect in one cycle and has priority over accumulation) producing two different behaviors at two different latencies. Our testbench will check that both versions of the MAC implement these behaviors correctly.

3.1. Implement and Simulate

Implement this pipelined MAC design in a file called `mac_pipe.sv` and name the top-level module `mac_pipe`. Otherwise, use the same parameters and port declarations given in Section 1.1.

Using the provided testbench (see Section 4), simulate your design thoroughly across the legal parameter ranges given in Section 1.1, just as you did for the unpipelined design in Section 2. Make sure you have thoroughly simulated both the unpipelined and pipelined designs.

3.2. Synthesize

Repeat the synthesis steps from Section 2.2 with `mac_pipe` (`WIDTH=16`, `ACCW=48`). You will need the results for Question 2. Include the synthesis logs with your Part 1 submission, and use the synthesis results to answer Question 2 in the Final Report.

4. Testbenches

We provide you with a large randomized testbench that can test many random inputs on any legal set of parameter values. This testbench uses a SystemVerilog module and an accompanying C program that the testbench runs via DPI.

You can find the testbench and some helpful scripts in the following directory:

```
/home/home4/pmilder/ese507/proj/part1/
```

Start by copying all files from that location into your `part1/` directory.

Before you start simulating, please read the following brief explanation of how the testbench works. Then read through the testbench files `mac_tb.sv` and `mac_tb.c` and their comments. (We also will discuss a testbench with similar structure in the Topic 8 slides.)

- The testbench has four parameters. Setting the `WIDTH` and `ACCW` parameters in the testbench will set the corresponding parameters in the MAC module. These are used when the testbench instantiates your `mac_pipe` or `mac` module.

The `TESTS` parameter chooses the number of random tests to run.

The PIPELINED parameter picks whether to simulate the unpipelined design (if PIPELINED==0) or the pipelined design (if PIPELINED==1).

- The testbench uses a SystemVerilog class called `testdata` to hold one cycle of test data. When a `testdata` object is randomized, it randomly chooses values of `input0`, `input1`, `input_valid`, `init_acc`, `init_value`, `reset`, and `Q`. The random values are not uniform: `init_acc` and `reset` are asserted only occasionally, and the data inputs are biased so that they take their most positive and most negative values more often than they otherwise would. (These extreme values are the ones that exercise saturation and the corners of the arithmetic.)
- The C program contains two functions which simulate the expected behavior for both versions of the MAC. For each cycle of test data, the testbench will call the appropriate C function to determine the expected result. Then, it checks that the simulated MAC output matches the expected output.
- Before the random tests, the testbench runs a short sequence of hand-chosen (directed) cycles that test saturation.
- The testbench will count the number of errors. It is set to stop running if the error count reaches 100. (You can easily modify this if you want to raise the limit.)

4.1. Using Scripts to Simplify Running Simulation

Since your design is parameterized, you need to run multiple simulations to check that it works for different parameter values. The testbench reads its parameter values from a file called `params.sv`, and we provide a script that generates that file, compiles everything, and runs the simulation in one step:

```
./simParams1 [WIDTH] [ACCW] [PIPELINED] [GUI]
```

Replace `[WIDTH]`, `[ACCW]`, and `[PIPELINED]` with the desired parameter values. Replace `[GUI]` with 0 to run in command line mode, or 1 to open the QuestaSim GUI so you can observe waveforms. For example, to simulate the unpipelined design with `WIDTH=8` and `ACCW=24` in command line mode, run:

```
./simParams1 8 24 0 0
```

To run the same simulation in GUI mode, add `&` to the end of the command so that the GUI runs in the background:

```
./simParams1 8 24 0 1 &
```

This is all you need to do to run the testbench; simply repeat it with different parameter values. (Note that the script compiles every `.sv` and `.c` file in the directory, so if your design uses additional `.sv` files, they will be included automatically.)

Random seeds. The `simParams1` script runs QuestaSim with the `-sv_seed random` flag, which initializes QuestaSim's random number generator differently on every run, so each run tests a different random sequence. Near the start of its output, the simulation prints the seed it chose in a line that looks like this (although the number will be different each time):

```
# Sv_Seed = 1896683400
```

This matters more than it may seem: when a random test finds a bug, you need to be able to reproduce the exact same sequence of inputs while you debug. To do that, you run the simulator manually with the seed from the failing run. First, generate `params.sv` (this is the first thing that `simParams1` does):

```
./genParams1 [WIDTH] [ACCW] [PIPELINED]
```

Then compile your code (adding any other `.sv` files your design uses):

```
vlog -64 +acc mac_tb.sv mac_tb.c mac.sv mac_pipe.sv
```

Finally, run QuestaSim with the seed you want to reproduce, in command line mode:

```
vsim -64 -c mac_tb -sv_seed 1896683400 -do "run -all; quit"
```

or in GUI mode:

```
vsim -64 mac_tb -sv_seed 1896683400 &
```

(Obviously, replace the number with the seed from your own simulation.)

5. Part 1 Submission

Your Part 1 submission should include your code for both designs, your synthesis reports, your AI usage log, and a brief plaintext status report. Please see Section 6 in the Project Overview document, which explains the submission requirements and procedure. In your `status.txt` file, be sure to explain how you tested your designs and what your testbench covers.

6. Final Report

Your Final Report is due at the end of the semester, although I recommend that you work on it as you complete each part of the project. In your Final Report, include a clearly labeled section for Part 1, where you answer the following questions. More information about the Final Report can be found in Section 6 in the Project Overview document.

1. Use Synopsys DesignCompiler to synthesize your unpipelined design with `WIDTH=16` and `ACCW=48` for a ***range of different clock frequencies from slow to fast***. Adapt the scripts you used in HW2.

For each frequency you try, record the area, power, the critical path location, and whether the timing constraint was met or violated. In your report, make a table that shows this data for each attempted frequency. Make sure you include units on all values you report (here and everywhere else in the report).

Make graphs that show the relationships you found between clock frequency and both area and power. Explain the trends that you observed and explain why they occur. (Make two graphs. On both, show clock frequency on the x-axis; then show area as the y-axis on one graph and power as the y-axis on the other.) Make sure you use graphs that plot both axes proportionally (like a scatter graph, not a line graph). Only include the design points where the timing constraint is MET.

For each frequency, give a description in your report of *where* the critical path is. Don't just copy/paste the endpoints from the synthesis report, but explain logically where the critical path lies in the module. (For example, "the critical path starts at the output of register [register name], passes through logic that computes [description of the logic], and ends at the input to register [register name].")

2. Now, repeat the tasks from question 1 for your pipelined design with the same values of WIDTH and ACCW. Additionally, answer the following: Did pipelining help make this module faster? Explain why or why not and show how this is reflected in the synthesis data and critical path.
3. For the pipelined design with the maximum clock frequency you found, how much energy would your system consume if it were to process a sequence of 50 *sets of input values*? Assume you have to wait until the final output comes out of the system, and don't forget that your pipelined design takes more than 50 cycles to compute 50 sets of inputs.

Remember: energy is measured in joules. Power = energy per time. 1 Watt = 1 Joule per second. Use the power obtained from synthesis and your understanding of the time it would take for your system to fully compute 50 sets of input values.

4. Would the energy you computed in question 3 change if you resynthesized the design targeting different clock frequencies? Explain and justify your answer. Think carefully about what changes when you change the target frequency and how those changes affect the power the system consumes.
5. Make a table that compares the power, area, latency, and throughput of your pipelined and unpipelined MAC designs (at the maximum clock frequency you previously found for each) with WIDTH=16 and ACCW=48. In your report show how you calculated the latency and throughput. Quantify latency in seconds (or ns), and quantify throughput in terms of MACs per second. (If needed, review these concepts in the Topic 6 slides.) Based on the trade-offs seen in your table, explain when it would make sense for a designer to choose the pipelined design and when it would make sense to use the unpipelined design.
6. Your design is pipelined as much as possible if you assume that you cannot pipeline the arithmetic units themselves. However, as we discussed in Topic 6, we could also pipeline the multiplier itself. For example, you can replace the multiplier with one that is pipelined into more stages. Based on your results to questions 1 and 2, would you expect that deeper pipelining in the multiplier might help you reach higher clock frequencies? Justify why or why not.

If you were to pipeline the multiplier in this way, what other changes would you have to make in your module? Be specific about each of the module's control inputs: which of them would need to change, and which would not?

Would pipelining inside of the adder be possible and a good idea? Why or why not?

(Answer this question based on your understanding of the design and your answers to prior questions. You do not need to modify your design/code to answer this question.)

7. In questions 1 and 2, you always synthesized using the same parameter values of WIDTH=16 and ACCW=48. Here, explore how changing WIDTH and ACCW affects the critical path location and maximum clock frequency of your pipelined MAC module. Don't forget: to change these parameters for synthesis, you should edit the default parameter values in your source code (mac_pipe.sv).

First, do a set of experiments where you set WIDTH=12 and synthesize three designs with ACCW=24, 48, and 64.

Next, do a new set of experiments where you set ACCW=48 and synthesize three designs with WIDTH=8, 16, 24.

Do the clock frequency and critical path location change as ACCW changes? Do they change when WIDTH changes? Explain what you see and what you learn from it. You should expect to see differences in the scaling behavior of WIDTH and ACCW. Explain why they behave differently. In your explanation, account for the shift-and-saturate logic on the output path as well as the multiplier and adder.

Include the six synthesis reports for this question along with your Final Report.

8. The MAC's accumulator holds ACCW bits. As you know, if the value stored in the accumulator grows large enough, it will overflow. That is, ACCW bits may not be enough to store the resulting number.

Assume WIDTH=5 and ACCW=16. What is the maximum number of MACs your system could perform while guaranteeing that the accumulator cannot overflow? (Hint: what is the largest magnitude number you could produce on the multiplier's output? Then, how many accumulations would it take for that number to produce an overflow in the accumulator?) Don't forget that our values are all signed integers, and don't forget that the accumulator can be initialized to a signed WIDTH-bit number. Show your reasoning and justify your answer.

9. If you worked with a partner, please carefully describe each partner's contribution to this part of the project. (If you did not work with a partner, skip this.)